

elements with a constant.

The corresponding program using NumPy would be:

```
import numpy as np
r = 1280*(1)
m = np.array(1024*(r))
print m.shape
for t in range(10):
    m2 = 5*m
```

As before, the array method converts a Python list into an array. Printing the shape of the array confirms that it is a two dimensional array of size 1024x1280. The core Python program takes 5.9 seconds while the NumPy application takes 0.7 seconds.

But notice how easy it is to multiply all elements in an array with a constant. You can use NumPy to define the matrix with all elements being 1, as follows:

```
import numpy as np
m = np.ones((1024,1280))
print m.shape
for t in range(10):
    mp = 5*m
```

This version runs in 0.4 seconds.

It is reasonable to assume that the addition of two arrays of the same shape is possible and the corresponding elements will be added. Explore the following:

```
>>> m = np.array([[1,2,3], [4,5,6]])
>>> n = np.array([[1,2,3], [4,5,6]])
>>> m.shape
(2, 3)
>>> n.shape
(2, 3)
>>> m + n
array([[ 2,  4,  6],
       [ 8, 10, 12]])
>>> n.shape = (6,1)
>>> m + n
```

You are creating two arrays of size 2x3 and adding them. The result is as expected. NumPy allows an array to be interpreted by changing the shape of the array. In

the example above, the array n is reinterpreted as a one-dimensional array. Now, if you try to add m and n, it should fail and it does.

Multiplication of two-dimensional arrays could be interpreted as a matrix multiplication. However, that would not be consistent across various dimensions. Hence, in NumPy, multiplication of two arrays is allowed if their shapes are identical and correspond to the creation of a new array whose elements are a product of the corresponding elements of the two arrays. You can try the following:

```
>>> m = np.array([[1,2],[3,4]])
>>> n = np.array([[1,2],[3,4]])
>>> m*n
array([[ 1,  4],
       [ 9, 16]])
```

If you want to work with matrices, you need to define a two-dimensional array as a matrix. Try the following example—compare and explore:

```
>>> m = np.matrix([[1,2],[3,4]])
>>> n = np.matrix([[1,2],[3,4]])
>>> m*n
matrix([[ 7, 10],
        [15, 22]])
>>> m.shape=4,1
>>> n.shape = 4,1
>>> m*n
>>> n*m
```

Use the built-in help to know more about the possibilities available for arrays, matrices and a lot more.

It should be clear that if an operation can be applied to an array, the performance gains over core Python could be substantial. NumPy provides a lot of functionality, which can be very useful when analysing large volumes of data, giving researchers an edge to beat the competition! 

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Tel.: 91-22-6633 8900 Ext. 324. info@technoinfotech.com